November 6, 2026 at 22:52

1. Introduction. A partial assignment is *stable* (or "valid") if it is consistent and unit propagation cannot extend it. Exercise 7.2.2.2–363 gives the stable partial assignments of a problem a weight, shows that they form a lattice, and asks ten questions about them. Its neighbor, exercise 364, sits on top of the same notion: a *covering* assignment is a stable one in which every assigned variable is constrained.

Two definitions do all the work, and both are quoted here as the exercise states them. A partial assignment is stable "if and only if no clause is entirely false, or entirely false except for at most one unassigned literal." And "variable x_k of a partial assignment is called *constrained* if it appears in a clause where $\pm x_k$ is true but all the other literals are false (thus its value has a 'reason')."

2. Writing $x \prec x'$ when x' agrees with x except that $x_k = *$ and $x'_k \in \{0, 1\}$, the exercise defines

$$x \sqsubseteq x' \quad \text{if} \quad x = x^{(0)} \prec x^{(1)} \prec \cdots \prec x^{(t)} = x' \text{ for some } t \geq 0,$$

where every $x^{(j)}$ is stable. Then, given probabilities $p_k + q_k = 1$, the weight of an unstable assignment is 0 and otherwise

$$W(x) = \prod\{p_k \mid x_k = *\} \cdot \prod\{q_k \mid x_k \neq * \text{ and } x_k \text{ is unconstrained}\}.$$

Everything here is finite and small: a problem on n variables has 3^n partial assignments, and the parts of the exercise ask about problems with three, four, or five of them. So this program tabulates all 3^n assignments, decides stability and constrainedness from the definitions, and answers each part by looking.

3. Part (a) is the exception. It asks whether the literals on Algorithm C's trail, at the moment step C5 chooses a new decision, always form a stable partial assignment—a question about a real solver, not about a table. The `sat` package of the directory above is a Go rendering of Knuth's `SAT13`, and it had no way to show its trail: the port deliberately leaves out the diagnostics that `SAT13` turns on with its `v` option, and `SAT13`'s own *sanity* routine, thorough as it is about *tloc* and *reason* and *value*, never looks at this. So the package grew a *SetTrace* hook while this reading was being written. It hands over the trail just before each decision and costs nothing when it isn't used, exactly as *SetProof* does.

Weights are kept as products of symbols rather than as numbers, so that the program's answer to part (e) can be compared with the book's table entry by entry. Only part (j), which is an identity about sums, needs arithmetic.

I wrote this on 16 September 2026.

4. Every part is a mode.

```
package main
import (
    "context"
    "flag"
    "fmt"
    "math/bits"
    "math/rand/v2"
    "os"
    "strconv"
    "strings"
    "github.com/sjnam/sat"
)
⟨ Types 9 ⟩
⟨ Global variables 6 ⟩
⟨ Functions 7 ⟩
func main() {
    ⟨ Read the command line 5 ⟩
    if on("trail") {
        ⟨ Check part (a) against Algorithm C 53 ⟩
    }
    if on("weights") {
        ⟨ Check parts (b) and (e) 19 ⟩
    }
    if on("stable") {
        ⟨ Check part (c) 23 ⟩
    }
    if on("climb") {
        ⟨ Check parts (d) and (f) 28 ⟩
    }
    if on("lattice") {
        ⟨ Check parts (h) and (i), and the lattice of part (d) 33 ⟩
    }
    if on("sum") {
        ⟨ Check the identity of part (j) 41 ⟩
    }
    if on("horn") {
        ⟨ Check the construction of part (g) 45 ⟩
    }
    ⟨ Say how it went 8 ⟩
}
```

5. ⟨ Read the command line 5 ⟩ ≡

```
flag.StringVar(&mode, "mode", "all",
    "trail, weights, stable, climb, lattice, sum, horn, or all")
flag.IntVar(&reps, "reps", 2000, "how many random cases per size")
flag.Uint64Var(&seed, "seed", 20260916, "seed for the random cases")
flag.Parse()
```

This code is used in section 4.

6. ⟨ Global variables 6 ⟩ ≡

```
var (
    mode string
    reps int
    seed uint64
    failures int
)
```

This code is also defined in sections 20, 29.

This code is used in section 4.

7. Each claim is announced, checked, and marked, and one `FAIL` anywhere makes the program exit with a nonzero status.

⟨ Functions 7 ⟩ ≡

```
func on(m string) bool { return mode ≡ m ∨ mode ≡ "all" }
func claim(ok bool, format string, args ... any) {
    mark := "ok␣␣"
    if ¬ok {
        mark = "FAIL"
        failures++
    }
    fmt.Printf("␣␣%s␣␣%s\n", mark, fmt.Sprintf(format, args...))
}
```

This code is also defined in sections 10, 12, 16, 18, 22, 26, 27, 31, 32, 40, 43, 47, 48, 52, 56, 57.

This code is used in section 4.

8. ⟨ Say how it went 8 ⟩ ≡

```
if failures > 0 {
    fmt.Printf("\n%d␣claims␣failed.\n", failures)
    os.Exit(1)
}
fmt.Println("\nEvery␣claim␣checked␣out.")
```

This code is used in section 4.

9. Clauses and partial assignments. The literal for variable v is $2v$ when positive and $2v+1$ when negative, as Knuth numbers them; a clause is a bitmask over literals and a family is a list of clauses. The notation is his too: '$12\bar{3}$' is the clause $(x_1 \vee x_2 \vee \bar{x}_3)$, which *parseFam* reads as `123'`.

⟨ Types 9 ⟩ ≡

```
type fam struct {
    n int
    cl []uint32
}
```

This code is also defined in sections 11, 17.

This code is used in section 4.

10. ⟨ Functions 7 ⟩ +≡

```
func lit(v int, positive bool) int {
    if positive {
        return 2 * v
    }
    return 2 * v + 1
}
func parseFam(n int, s string) fam {
    f := fam{n: n}
    for _, w := range strings.Fields(s) {
        var c uint32
        for i := 0; i < len(w); i++ {
            v := int(w[i] - '0')
            positive := true
            if i + 1 < len(w) ∧ w[i + 1] ≡ '\'' {
                positive, i = false, i + 1
            }
            c |= 1 ≪ lit(v, positive)
        }
        f.cl = append(f.cl, c)
    }
    return f
}
```

11. A partial assignment is written in base three, digit $v-1$ saying whether variable v is unassigned, true, or false. The program tabulates, for every one of the 3^n of them, which literals it makes true, which false, which are still free, whether it is stable, and which variables it constrains.

⟨ Types 9 ⟩ +≡

```
type ctx struct {
    f fam
    n, nA int
    pow3 []int
    tmask []uint32   // the literals made true
    fmask []uint32   // the literals made false
    free []uint32    // the literals on unassigned variables
    isStable []bool
    cons []uint32    // bit v: the assignment constrains variable v
}
```

12. ⟨ Functions 7 ⟩ +≡

```
func newCtx(f fam) *ctx {
    c := &ctx{f: f, n: f.n}
    ⟨ Set up the powers of three 13 ⟩
    ⟨ Tabulate the masks 14 ⟩
    ⟨ Decide stability and constrainedness 15 ⟩
    return c
}
func (c *ctx) digit(a, v int) int { return (a/c.pow3[v − 1]) % 3 }
```

13. ⟨ Set up the powers of three 13 ⟩ ≡

```
c.pow3 = make([]int, c.n + 1)
c.pow3[0] = 1
for i := 1; i ≤ c.n; i++ {
    c.pow3[i] = 3 * c.pow3[i − 1]
}
c.nA = c.pow3[c.n]
```

This code is used in section 12.

14. ⟨ Tabulate the masks 14 ⟩ ≡

```
c.tmask = make([]uint32, c.nA)
c.fmask = make([]uint32, c.nA)
c.free = make([]uint32, c.nA)
c.isStable = make([]bool, c.nA)
c.cons = make([]uint32, c.nA)
for a := 0; a < c.nA; a++ {
    for v := 1; v ≤ c.n; v++ {
        switch c.digit(a, v) {
        case 0:
            c.free[a] |= 1 ≪ lit(v, true) | 1 ≪ lit(v, false)
        case 1:
            c.tmask[a] |= 1 ≪ lit(v, true)
            c.fmask[a] |= 1 ≪ lit(v, false)
        case 2:
            c.tmask[a] |= 1 ≪ lit(v, false)
            c.fmask[a] |= 1 ≪ lit(v, true)
        }
    }
}
```

This code is used in section 12.

15. Straight from the two definitions. A clause that is still unsatisfied leaves the assignment stable only if two or more of its literals are unassigned; a clause that is satisfied by exactly one of its literals, all the others being false, constrains that literal's variable.

⟨ Decide stability and constrainedness 15 ⟩ ≡

```
for a := 0; a < c.nA; a++ {
    c.isStable[a] = true
    for _, cl := range c.f.cl {
        if t := cl & c.tmask[a]; t ≠ 0 {
            if bits.OnesCount32(t) ≡ 1 ∧ cl ⊘ (c.fmask[a] | t) ≡ 0 {
                c.cons[a] |= 1 ≪ uint(bits.TrailingZeros32(t) ≫ 1)
            }
        } else if bits.OnesCount32(cl ⊘ c.fmask[a]) ≤ 1 {
            c.isStable[a] = false
        }
    }
}
```

This code is used in section 12.

16. Two conveniences: *assign* sets the variable of a literal so that the literal becomes true, and *code* reads an assignment written as a string like `10*`.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) assign(a, l int) int {
    v := l ≫ 1
    if c.digit(a, v) ≠ 0 {
        return −1
    }
    if l & 1 ≡ 0 {
        return a + c.pow3[v − 1]
    }
    return a + 2 * c.pow3[v − 1]
}
func (c *ctx) code(s string) int {
    a := 0
    for i := 0; i < len(s) ∧ i < c.n; i++ {
        switch s[i] {
        case '1':
            a += c.pow3[i]
        case '0':
            a += 2 * c.pow3[i]
        }
    }
    return a
}
```

17. Weights. A weight is a product of p's and q's, so the program keeps it as two bitmasks and prints it the way the book does: p for the unassigned variables, q for the assigned ones that are unconstrained, in order of the variables, with the empty product printed as 1.

⟨ Types 9 ⟩ +≡

```
type term struct {
    p, q uint32
    zero bool
}
```

18. ⟨ Functions 7 ⟩ +≡

```
func (c *ctx) weight(a int) term {
    if ¬c.isStable[a] {
        return term{zero: true}
    }
    var t term
    for v := 1; v ≤ c.n; v++ {
        switch {
        case c.digit(a, v) ≡ 0:
            t.p |= 1 ≪ v
        case c.cons[a] & (1 ≪ v) ≡ 0:
            t.q |= 1 ≪ v
        }
    }
    return t
}
func (t term) String() string {
    if t.zero {
        return "0"
    }
    var b strings.Builder
    for v := 1; v < 32; v++ {
        if t.p & (1 ≪ v) ≠ 0 {
            b.WriteString("p" + strconv.Itoa(v))
        }
        if t.q & (1 ≪ v) ≠ 0 {
            b.WriteString("q" + strconv.Itoa(v))
        }
    }
    if b.Len() ≡ 0 {
        return "1"
    }
    return b.String()
}
```

19. Part (b) asks for the weights of the clauses F of 7.2.2.2–(1), which are $\{1\bar2, 23, \bar1\bar3, \bar1\bar23\}$. The answer says $W(001) = 1$, $W(***) = p_1p_2p_3$, and $W(x) = 0$ otherwise—so all three variables are constrained in the one solution, and the empty assignment is the only other stable one.

```
⟨ Check parts (b) and (e) 19 ⟩ ≡
    fmt.Println("363(b). the weights of the clauses F of (1)")
    b := newCtx(parseFam(3, "12' 23 1'3' 1'2'3"))
    ok := b.weight(b.code("001")).String( ) ≡ "1" ∧
        b.weight(b.code("***")).String( ) ≡ "p1p2p3"
    for a := 0; a < b.nA; a++ {
        if a ≠ b.code("001") ∧ a ≠ b.code("***") ∧ ¬b.weight(a).zero {
            ok = false
        }
    }
    claim(ok, "W(001) = 1, W(***) = p1p2p3, and every other weight is 0")
    ⟨ Check the table of part (e) 21 ⟩
```

This code is used in section 4.

20. Part (e) asks for the weights when the only clause is 123. The answer gives them as a table with a row for each value of x_1 and a column for each value of x_2x_3, and here it is, copied from page 619.

```
⟨ Global variables 6 ⟩ +≡
    var (
        cols = [9]string{"00", "01", "0*", "10", "11", "1*", "*0", "*1", "**"}
        rows = [3]string{"0", "1", "*"}
        book = [3][9]string{
            {"0", "q1q2", "0", "q1q3", "q1q2q3", "q1q2p3", "0", "q1p2q3", "q1p2p3"},
            {"q2q3", "q1q2q3", "q1q2p3", "q1q2q3", "q1q2q3", "q1q2p3",
                "q1p2q3", "q1p2q3", "q1p2p3"},
            {"0", "p1q2q3", "p1q2p3", "p1q2q3", "p1q2q3", "p1q2p3",
                "p1p2q3", "p1p2q3", "p1p2p3"},
        }
    )
```

21.

```
⟨ Check the table of part (e) 21 ⟩ ≡
    fmt.Println("363(e). the weights when the only clause is 123")
    e := newCtx(parseFam(3, "123"))
    bad := 0
    for i, r := range rows {
        line := "         x1 = " + r + ":"
        for j, col := range cols {
            got := e.weight(e.code(r + col)).String( )
            if got ≠ book[i][j] {
                bad++
                got += "(book " + book[i][j] + ")"
            }
            line += fmt.Sprintf(" %8s", got)
        }
        fmt.Println(line)
    }
    claim(bad ≡ 0, "all 27 weights agree with the table on page 619")
```

This code is used in section 19.

22. Stability, one variable at a time. Part (c) takes a stable x with $x_k = 1$ and forms x' by setting $x_k \leftarrow 0$ and x'' by setting $x_k \leftarrow *$. It then asks whether "x_k is unconstrained in x" is the same as (i) x' is consistent; (ii) x' is stable; (iii) x'' is stable. The answer: (i) and (iii) yes, (ii) no, and the counterexample offered is $x = 10*$ with $x' = 00*$, against the clause 123.

Consistency is the first half of stability—no clause entirely false—so the program needs it separately.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) consistent(a int) bool {
    for _, cl := range c.f.cl {
        if cl & c.tmask[a] ≡ 0 ∧ cl ⊘ c.fmask[a] ≡ 0 {
            return false
        }
    }
    return true
}
```

23. ⟨ Check part (c) 23 ⟩ ≡

```
fmt.Println("363(c). complementing or erasing one value")
var n3, bad1, bad2, bad3 int
sweep(4, reps, func(c *ctx) {
    for a := 0; a < c.nA; a++ {
        for v := 1; v ≤ c.n; v++ {
            if ¬c.isStable[a] ∨ c.digit(a, v) ≠ 1 {
                continue
            }
            n3++
            ⟨ Compare the three statements with unconstrainedness 24 ⟩
        }
    }
})
claim(bad1 ≡ 0, "(i) x' is consistent: %d cases, %d exceptions", n3, bad1)
claim(bad3 ≡ 0, "(iii) x'' is stable: %d exceptions", bad3)
claim(bad2 > 0, "(ii) x' is stable: false, %d exceptions", bad2)
⟨ Check the counterexample the answer gives 25 ⟩
```

This code is used in section 4.

24. ⟨ Compare the three statements with unconstrainedness 24 ⟩ ≡

```
free := c.cons[a] & (1 ≪ v) ≡ 0
xp := a − c.pow3[v − 1] + 2 ∗ c.pow3[v − 1]
xpp := a − c.pow3[v − 1]
if free ≠ c.consistent(xp) {
    bad1++
}
if free ≠ c.isStable[xp] {
    bad2++
}
if free ≠ c.isStable[xpp] {
    bad3++
}
```

This code is used in section 23.

25. ⟨ Check the counterexample the answer gives 25 ⟩ ≡

```
one := newCtx(parseFam(3, "123"))
x, xp := one.code("10*"), one.code("00*")
claim(one.isStable[x] ∧ one.cons[x] & (1 ≪ 1) ≡ 0 ∧ one.consistent(xp) ∧
    ¬one.isStable[xp], "␣␣␣␣␣and␣x␣=␣10*,␣x'␣=␣00*␣with␣the␣clause␣123␣is␣one")
```

This code is used in section 23.

26. Climbing. The relation $x \sqsubseteq x'$ asks for a chain of stable assignments from x up to x', one literal at a time. Since every step adds a literal, the assignments x with $x \sqsubseteq x'$ are found by walking down from x', removing one literal at a time and never leaving stability.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) below(a int) []bool {
    seen := make([]bool, c.nA)
    if ¬c.isStable[a] {
        return seen
    }
    seen[a] = true
    for queue := []int { a }; len(queue) > 0; {
        b := queue[len(queue) − 1]
        queue = queue[:len(queue) − 1]
        for v := 1; v ≤ c.n; v++ {
            if d := c.digit(b, v); d ≠ 0 {
                e := b − d ∗ c.pow3[v − 1]
                if c.isStable[e] ∧ ¬seen[e] {
                    seen[e], queue = true, append(queue, e)
                }
            }
        }
    }
    return seen
}
```

27. It is convenient to name an assignment by the set of literals it holds, the way the exercise does.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) name(a int) string {
    var b strings.Builder
    for v := 1; v ≤ c.n; v++ {
        if d := c.digit(a, v); d ≠ 0 {
            if b.Len( ) > 0 {
                b.WriteByte(',')
            }
            b.WriteString(strconv.Itoa(v))
            if d ≡ 2 {
                b.WriteByte('\'')
            }
        }
    }
    return "{" + b.String( ) + "}"
}
```

28. Part (d) asks for all L with $L \sqsubseteq \{1, \bar{2}, \bar{3}\}$ when the only clause is 123. The answer: all eight subsets of $\{1, \bar{2}, \bar{3}\}$ are stable except $\{\bar{2}, \bar{3}\}$, where x_1 is constrained, so seven remain.

⟨ Check parts (d) and (f) 28 ⟩ ≡

```
fmt.Println("363(d). climbing to {1,2',3'} with the single clause 123")
d := newCtx(parseFam(3, "123"))
target := d.code("100")
seen, got := d.below(target), 0
for a := 0; a < d.nA; a++ {
    if seen[a] {
        got++
    }
}
claim(got ≡ 7 ∧ ¬d.isStable[d.code("*00")],
    "seven of the eight subsets, all but {2',3'}, which is not stable")
⟨ Check the twelve sets of part (f) 30 ⟩
```

This code is used in section 4.

29. Part (f) wants clauses whose $L \sqsubseteq \{1, 2, 3, 4, 5\}$ are twelve given sets. The answer offers $\{\bar{1}\bar{2}3\bar{4}\bar{5}, \bar{1}4, \bar{2}5, \bar{3}\bar{4}\bar{5}, \bar{3}4\bar{5}\}$, and adds that $\{3\}$ is stable but unreachable below $\{1, 2, 3, 4, 5\}$.

⟨ Global variables 6 ⟩ +≡

```
var twelve = []string{"", "4", "5", "14", "25", "45", "145", "245", "345",
    "1345", "2345", "12345"}
```

30. ⟨ Check the twelve sets of part (f) 30 ⟩ ≡

```
fmt.Println("363(f). twelve sets below {1,2,3,4,5}")
f := newCtx(parseFam(5, "1'2'34'5' 1'4 2'5 3'45' 3'4'5"))
want := make([]bool, f.nA)
for _, s := range twelve {
    want[setCode(f, s)] = true
}
below := f.below(setCode(f, "12345"))
diff := 0
for a := 0; a < f.nA; a++ {
    if below[a] ≠ want[a] {
        diff++
    }
}
claim(diff ≡ 0, "the answer's clauses give exactly the twelve sets asked for")
claim(f.isStable[setCode(f, "3")] ∧ ¬below[setCode(f, "3")],
    "     and {3} is stable but unreachable, as the answer remarks")
```

This code is used in section 28.

31. ⟨ Functions 7 ⟩ +≡

```
func setCode(c *ctx, s string) int {
    a := 0
    for i := 0; i < len(s); i++ {
        a = c.assign(a, lit(int(s[i] - '0'), true))
    }
    return a
}
```

32. The lattice. Part (h) says that if L, L', L'' are stable with $L' \prec L$ and $L'' \prec L$, then $L' \cap L''$ is stable; part (i) says that $L' \sqsubseteq L$ and $L'' \sqsubseteq L$ imply $L' \cap L'' \sqsubseteq L$. Intersecting two assignments means keeping the values they agree on, which in base three is done digit by digit.

```
⟨ Functions 7 ⟩ +≡
    func (c *ctx) meet(a, b int) int {
        m := 0
        for v := 1; v ≤ c.n; v++ {
            if d := c.digit(a, v); d ≡ c.digit(b, v) {
                m += d * c.pow3[v − 1]
            }
        }
        return m
    }
```

33.

```
⟨ Check parts (h) and (i), and the lattice of part (d) 33 ⟩ ≡
    fmt.Println("363(h,i). intersections of stable assignments")
    var nh, badH, badI int
    sweep(4, reps, func(c *ctx) {
        for a := 0; a < c.nA; a++ {
            if ¬c.isStable[a] {
                continue
            }
            ⟨ Check part (h) at a 34 ⟩
        }
    })
    sweep(4, reps/50, func(c *ctx) {
        ⟨ Check part (i) at every target 35 ⟩
    })
    claim(badH ≡ 0, "(h) holds: %d pairs one step below a stable L, %d exceptions",
        nh, badH)
    claim(badI ≡ 0, "(i) holds: %d exceptions", badI)
    ⟨ Look at the lattice that part (d) mentions 36 ⟩
```

This code is used in section 4.

34. ⟨ Check part (h) at a 34 ⟩ ≡

```
for v := 1; v ≤ c.n; v++ {
    for w := v + 1; w ≤ c.n; w++ {
        dv, dw := c.digit(a, v), c.digit(a, w)
        if dv ≡ 0 ∨ dw ≡ 0 {
            continue
        }
        lp, lpp := a − dv ∗ c.pow3[v − 1], a − dw ∗ c.pow3[w − 1]
        if ¬c.isStable[lp] ∨ ¬c.isStable[lpp] {
            continue
        }
        nh++
        if ¬c.isStable[c.meet(lp, lpp)] {
            badH++
        }
    }
}
```

This code is used in section 33.

35. ⟨ Check part (i) at every target 35 ⟩ ≡

```
for a := 0; a < c.nA; a++ {
    if ¬c.isStable[a] {
        continue
    }
    seen := c.below(a)
    for x := 0; x < c.nA; x++ {
        if ¬seen[x] {
            continue
        }
        for y := x + 1; y < c.nA; y++ {
            if seen[y] ∧ ¬seen[c.meet(x, y)] {
                badI++
            }
        }
    }
}
```

This code is used in section 33.

36. The seven sets of part (d), ordered by inclusion, are said to illustrate L_7, "the smallest lattice that is lower semimodular but not modular." The program collects them and checks all three words.

⟨ Look at the lattice that part (d) mentions 36 ⟩ ≡

```
fmt.Println("363(d). the shape of those seven sets")
l7 := newCtx(parseFam(3, "123"))
var elems []int
for a, in := range l7.below(l7.code("100")) {
    if in {
        elems = append(elems, a)
    }
}
⟨ Decide whether the elements form a lattice 37 ⟩
claim(len(elems) ≡ 7 ∧ isLattice, "the seven sets form a lattice")
claim(lowerSemi ∧ ¬modular, "it is lower semimodular but not modular")
```

This code is used in section 33.

37. Within this family the meet is the intersection and the join is the smallest member containing both, when there is one. The order is inclusion of the literal sets, which for these all-assigned-or-unassigned members is just $meet(a, b) \equiv a$.

⟨ Decide whether the elements form a lattice 37 ⟩ ≡

```
isLattice, lowerSemi, modular := true, true, true
join := map[[2]int]int{}
for _, a := range elems {
    for _, b := range elems {
        ⟨ Find the join of a and b 38 ⟩
    }
}
if isLattice {
    ⟨ Test lower semimodularity and modularity 39 ⟩
}
```

This code is used in section 36.

38. ⟨ Find the join of a and b 38 ⟩ ≡

```
best := −1
for _, u := range elems {
    if l7.meet(u, a) ≡ a ∧ l7.meet(u, b) ≡ b {
        if best < 0 ∨ l7.meet(best, u) ≡ u {
            best = u
        }
    }
}
if best < 0 {
    isLattice = false
} else {
    for _, u := range elems {
        if l7.meet(u, a) ≡ a ∧ l7.meet(u, b) ≡ b ∧ l7.meet(best, u) ≠ best {
            isLattice = false
        }
    }
    join[[2]int{a, b}] = best
}
```

This code is used in section 37.

39. A lattice is lower semimodular when $a \vee b$ covering both a and b forces a and b to cover $a \wedge b$; it is modular when $a \leq c$ implies $a \vee (b \wedge c) = (a \vee b) \wedge c$.

⟨ Test lower semimodularity and modularity 39 ⟩ ≡

```
covers := func(u, v int) bool {
    if u ≡ v ∨ l7.meet(u, v) ≠ v {
        return false
    }
    for _, w := range elems {
        if w ≠ u ∧ w ≠ v ∧ l7.meet(u, w) ≡ w ∧ l7.meet(w, v) ≡ v {
            return false
        }
    }
    return true
}
for _, a := range elems {
    for _, b := range elems {
        j, m := join([2]int{a, b}], l7.meet(a, b)
        if covers(j, a) ∧ covers(j, b) ∧ ¬(covers(a, m) ∧ covers(b, m)) {
            lowerSemi = false
        }
        for _, cc := range elems {
            if l7.meet(a, cc) ≠ a {
                continue
            }
            lhs := join[[2]int{a, l7.meet(b, cc)}]
            if lhs ≠ l7.meet(join[[2]int{a, b}], cc) {
                modular = false
            }
        }
    }
}
```

This code is used in section 37.

40. The identity of part (j). Part (j) claims that $\sum_{x' \sqsubseteq x} W(x') = \prod\{p_k \mid x_k = *\}$ whenever x is stable—the sum running over the assignments *below* x. With the single clause 123 and $x = 001$ the sum has seven terms and comes to 1; the program checks the identity for every stable assignment of random problems, with random probabilities.

⟨ Functions 7 ⟩ +≡

```
func (t term) value(p []float64) float64 {
    if t.zero {
        return 0
    }
    x := 1.0
    for v := 1; v < len(p); v++ {
        if t.p & (1 << v) ≠ 0 {
            x *= p[v]
        }
        if t.q & (1 << v) ≠ 0 {
            x *= 1 − p[v]
        }
    }
    return x
}
```

41. ⟨ Check the identity of part (j) 41 ⟩ ≡

```
fmt.Println("363(j). the sum of the weights below a stable assignment")
var nj, badJ int
rng := rand.New(rand.NewPCG(seed, 363))
sweep(4, reps/4, func(c *ctx) {
    p := make([]float64, c.n + 1)
    for v := 1; v ≤ c.n; v++ {
        p[v] = rng.Float64( )
    }
    for a := 0; a < c.nA; a++ {
        if ¬c.isStable[a] {
            continue
        }
        nj++
        ⟨ Compare the sum below a with the product of its p's 42 ⟩
    }
})
claim(badJ ≡ 0, "%d stable assignments, %d where the identity fails", nj, badJ)
```

This code is used in section 4.

42. ⟨ Compare the sum below a with the product of its p's 42 ⟩ ≡

```
sum := 0.0
for x, in := range c.below(a) {
    if in {
        sum += c.weight(x).value(p)
    }
}
want := 1.0
for v := 1; v ≤ c.n; v++ {
    if c.digit(a, v) ≡ 0 {
        want *= p[v]
    }
}
if diff := sum − want; diff > 1e − 9 ∨ diff < −1e − 9 {
    badJ++
}
```

This code is used in section 41.

43. Definite Horn clauses for a given family. Part (g) starts from a family $\mathcal{L}$ of subsets of $\{1,\ldots,n\}$, closed under intersection, in which every member can climb to $\{1,\ldots,n\}$ inside $\mathcal{L}$. It asks for definite Horn clauses—exactly one positive literal each—with $L \in \mathcal{L}$ if and only if $L \sqsubseteq \{1,\ldots,n\}$.

The answer's recipe is to look at the places where the family stops: if $L = L' \setminus l$ with $L' \in \mathcal{L}$ but $L \notin \mathcal{L}$, introduce a clause that makes L unstable by forcing l. Such a clause is $(x_l \vee \bigvee \bar{x}_k)$, and everything turns on the range of that disjunction.

⟨ Functions 7 ⟩ +≡

```
func horn(n int, ell map[uint32]bool, printed bool) fam {
    f := fam{n: n}
    for lp := range ell {
        for v := 1; v ≤ n; v++ {
            if lp & (1 ≪ v) ≡ 0 ∨ ell[lp ⊘ (1 ≪ v)] {
                continue
            }
            src := lp ⊘ (1 ≪ v)
            if printed {
                src = lp
            }
            ⟨ Add the clause that forces v 44 ⟩
        }
    }
    return f
}
```

44. ⟨ Add the clause that forces v 44 ⟩ ≡

```
cl := uint32(1) ≪ lit(v, true)
for k := 1; k ≤ n; k++ {
    if src & (1 ≪ k) ≠ 0 {
        cl |= 1 ≪ lit(k, false)
    }
}
f.cl = append(f.cl, cl)
```

This code is used in section 43.

45. The book's disjunction runs over $k \in L'$. But l belongs to L', so the clause then contains x_l and $\bar{x}_l$ both: it is a tautology, which is no clause at all in Knuth's convention and can never force anything. Ranging over $k \in L$ instead gives the implication $\bigwedge_{k\in L} x_k \Rightarrow x_l$, which makes L unstable exactly as wanted. The program builds both and compares.

⟨ Check the construction of part (g) 45 ⟩ ≡

```
fmt.Println("363(g). definite Horn clauses for a family of sets")
⟨ Try both readings on the family of part (f) 46 ⟩
⟨ Try both readings on random families 49 ⟩
```

This code is used in section 4.

46. ⟨ Try both readings on the family of part (f) 46 ⟩ ≡

```
ell := map[uint32]bool{}
for _, s := range twelve {
    var m uint32
    for i := 0; i < len(s); i++ {
        m |= 1 << uint(s[i] - '0')
    }
    ell[m] = true
}
claim(closedUnderMeet(ell) ∧ climbs(5, ell),
    "the twelve sets of part (f) are closed under intersection and climb")
claim(rebuilds(5, ell, false), "over k in L, the %d clauses rebuild the family",
    len(horn(5, ell, false).cl))
claim(¬rebuilds(5, ell, true), "over k in L', as printed, they do not")
```

This code is used in section 45.

47. *rebuilds* runs the construction and asks whether the sets below $\{1, \ldots, n\}$ are the family we started from.

⟨ Functions 7 ⟩ +≡

```
func rebuilds(n int, ell map[uint32]bool, printed bool) bool {
    c := newCtx(horn(n, ell, printed))
    all := 0
    for v := 1; v ≤ n; v++ {
        all = c.assign(all, lit(v, true))
    }
    below := c.below(all)
    for a := 0; a < c.nA; a++ {
        var m uint32
        positive := true
        for v := 1; v ≤ n; v++ {
            switch c.digit(a, v) {
            case 1:
                m |= 1 << v
            case 2:
                positive = false
            }
        }
        if positive ∧ below[a] ≠ ell[m] {
            return false
        }
        if ¬positive ∧ below[a] {
            return false
        }
    }
    return true
}
```

48. ⟨ Functions 7 ⟩ +≡

```
func closedUnderMeet(ell map[uint32]bool) bool {
    for a := range ell {
        for b := range ell {
            if ¬ell[a & b] {
                return false
            }
        }
    }
    return true
}
func climbs(n int, ell map[uint32]bool) bool {
    up := map[uint32]bool{(1 ≪ uint(n + 1)) − 2: true}
    for again := true; again; {
        again = false
        for a := range ell {
            if up[a] {
                continue
            }
            for v := 1; v ≤ n; v++ {
                if a & (1 ≪ v) ≡ 0 ∧ up[a | 1 ≪ v] {
                    up[a], again = true, true
                }
            }
        }
    }
    for a := range ell {
        if ¬up[a] {
            return false
        }
    }
    return true
}
```

49. Random families are made by taking a random collection of subsets, throwing in $\{1, \ldots, n\}$, closing it under intersection, and then dropping whatever cannot climb—repeating until nothing changes, which leaves a family that meets the hypotheses of part (g).

⟨ Try both readings on random families 49 ⟩ ≡

```
var nfam, badNarrow, badPrinted int
rg := rand.New(rand.NewPCG(seed, 364))
for i := 0; i < reps/4; i++ {
    n := 3 + rg.IntN(2)
    ⟨ Build a random family that satisfies the hypotheses 50 ⟩
    if len(horn(n, ell, false).cl) ≡ 0 {
        continue   // the whole power set; the construction has nothing to do
    }
    nfam++
    if ¬rebuilds(n, ell, false) {
        badNarrow++
    }
    if ¬rebuilds(n, ell, true) {
        badPrinted++
    }
}
claim(badNarrow ≡ 0, "over␣k␣in␣L:␣%d␣random␣families␣that␣need␣clauses,␣" +
    "%d␣failures", nfam, badNarrow)
claim(badPrinted ≡ nfam, "over␣k␣in␣L',␣as␣printed:␣all␣%d␣of␣them␣fail",
    nfam)
```

This code is used in section 45.

50. ⟨ Build a random family that satisfies the hypotheses 50 ⟩ ≡

```
ell := map[uint32]bool{(1 ≪ uint(n + 1)) − 2: true}
for j, k := 0, 2 + rg.IntN(2); j < k; j++ {
    var m uint32
    for v := 1; v ≤ n; v++ {
        if rg.IntN(2) ≡ 1 {
            m |= 1 ≪ v
        }
    }
    ell[m] = true
}
for again := true; again; {
    again = false
    for a := range ell {
        for b := range ell {
            if ¬ell[a & b] {
                ell[a & b], again = true, true
            }
        }
    }
    if ¬climbs(n, ell) {
        ⟨ Drop the members that cannot climb 51 ⟩
        again = true
    }
}
```

This code is used in section 49.

51. ⟨ Drop the members that cannot climb 51 ⟩ ≡

```
up := map[uint32]bool{(1 ≪ uint(n + 1)) − 2: true}
for more := true; more; {
    more = false
    for a := range ell {
        if up[a] {
            continue
        }
        for v := 1; v ≤ n; v++ {
            if a & (1 ≪ v) ≡ 0 ∧ up[a | 1 ≪ v] {
                up[a], more = true, true
            }
        }
    }
}
for a := range ell {
    if ¬up[a] {
        delete(ell, a)
    }
}
```

This code is used in section 50.

52. Algorithm C's trail. Part (a) is about the solver itself: is the partial assignment on the trail, at the moment step C5 picks a new decision literal, always stable? The answer says yes, and calls it an important invariant of Algorithm C.

The package hands the trail over through *SetTrace*. The program gives the solver clauses it has built itself, so it can check each trail against those same clauses, from the definition, with no help from the solver.

⟨ Functions 7 ⟩ +≡

```
func trailStable(f fam, trail []sat.Lit) bool {
    var tm, fm uint32
    for _, l := range trail {
        tm |= 1 ≪ uint(l)
        fm |= 1 ≪ uint(l ⊕ 1)
    }
    for _, cl := range f.cl {
        if cl & tm ≡ 0 ∧ bits.OnesCount32(cl ⊘ fm) ≤ 1 {
            return false
        }
    }
    return true
}
```

53. Two kinds of problem are tried: random 3SAT at a ratio that makes both answers common, and the van der Waerden clauses that open the section, which are unsatisfiable for $n \geq 9$ and make Algorithm C work.

⟨ Check part (a) against Algorithm C 53 ⟩ ≡

```
fmt.Println("363(a). the trail whenever Algorithm C decides")
var runs, decisions, unstable int
rt := rand.New(rand.NewPCG(seed, 5))
for i := 0; i < reps/20; i++ {
    ⟨ Make a random 3SAT problem 54 ⟩
    ⟨ Solve it, watching every decision 55 ⟩
}
for n := 9; n ≤ 20; n++ {
    f := waerden(n)
    ⟨ Solve it, watching every decision 55 ⟩
}
claim(unstable ≡ 0, "%d runs, %d decisions, %d unstable trails",
    runs, decisions, unstable)
```

This code is used in section 4.

54. ⟨ Make a random 3SAT problem 54 ⟩ ≡

```
n := 12 + rt.IntN(12)
f := fam{n: n}
for j := 0; j < 4 * n; j++ {
    var cl uint32
    for bits.OnesCount32(cl) < 3 {
        v := 1 + rt.IntN(n)
        if cl & (3 << lit(v, true)) == 0 {
            cl |= 1 << lit(v, rt.IntN(2) == 1)
        }
    }
    f.cl = append(f.cl, cl)
}
```

This code is used in section 53.

55. The variables are created in order, so the solver numbers them the way this program does, and a literal of the solver is a literal here.

⟨ Solve it, watching every decision 55 ⟩ ≡

```
s := sat.New()
for v := 1; v ≤ f.n; v++ {
    s.Lookup(strconv.Itoa(v))
}
for _, cl := range f.cl {
    var ls []sat.Lit
    for l := 2; l < 2 * f.n + 2; l++ {
        if cl & (1 << l) ≠ 0 {
            ls = append(ls, sat.Lit(l))
        }
    }
    s.AddClause(ls...)
}
s.SetTrace(func(level int, trail []sat.Lit) {
    decisions++
    if ¬trailStable(f, trail) {
        unstable++
    }
})
if _, err := s.Solve(context.Background()); err ≠ Λ {
    claim(false, "the␣solver␣failed:␣%v", err)
}
runs++
```

This code is used in section 53.

56. The clauses $waerden(3, 3; n)$ ask for a binary string of length n with no three equally spaced 0s and no three equally spaced 1s.

⟨ Functions 7 ⟩ +≡

```
func waerden(n int) fam {
    f := fam{n: n}
    for d := 1; d ≤ (n − 1)/2; d++ {
        for i := 1; i + 2 ∗ d ≤ n; i++ {
            var pos, neg uint32
            for j := 0; j < 3; j++ {
                pos |= 1 ≪ lit(i + j ∗ d, true)
                neg |= 1 ≪ lit(i + j ∗ d, false)
            }
            f.cl = append(f.cl, pos, neg)
        }
    }
    return f
}
```

57. Random problems. The parts that speak of all families are checked on random ones, a few thousand per number of variables. A family is a handful of clauses drawn from the 3^n clauses on n variables, the empty clause included, since nothing in the exercise rules it out.

⟨ Functions 7 ⟩ +≡

```
func sweep(maxN, count int, do func(*ctx)) {
    for n := 1; n ≤ maxN; n++ {
        cands := allClauses(n)
        rng := rand.New(rand.NewPCG(seed, uint64(n)))
        for i := 0; i < count; i++ {
            f, seen := fam{n: n}, map[uint32]bool{}
            for j, m := 0, 1 + rng.IntN(8); j < m; j++ {
                if cl := cands[rng.IntN(len(cands))]; ¬seen[cl] {
                    seen[cl] = true
                    f.cl = append(f.cl, cl)
                }
            }
            do(newCtx(f))
        }
    }
}
func allClauses(n int) []uint32 {
    out := []uint32{0}
    for v := 1; v ≤ n; v++ {
        next := make([]uint32, 0, 3 * len(out))
        for _, c := range out {
            next = append(next, c, c | 1 ≪ lit(v, true), c | 1 ≪ lit(v, false))
        }
        out = next
    }
    return out
}
```

58. Index.

a: 12, 14, 15, 16, 18, 19, 22, 23, 24, 26, 27, 28, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 41, 42, 47, 48, 50, 51.
AddClause: 55.
again: 48, 50.
all: 47.
allClauses: 57.
append: 10, 26, 36, 44, 54, 55, 56, 57.
args: 7.
assign: 16, 31, 47.
b: 18, 19, 26, 27, 32, 37, 38, 39, 48, 50.
Background: 55.
bad: 21.
badH: 33, 34.
badI: 33, 35.
badJ: 41, 42.
badNarrow: 49.
badPrinted: 49.
bad1: 23, 24.
bad2: 23, 24.
bad3: 23, 24.
below: 26, 28, 30, 35, 36, 42, 47.
best: 38.
bits: 15, 52, 54.
book: 20, 21.
Builder: 18, 27.
c: 10, 12, 13, 14, 15, 16, 18, 22, 23, 24, 26, 27, 31, 32, 33, 34, 35, 41, 42, 47, 57.
cands: 57.
cc: 39.
cl: 9, 10, 15, 22, 44, 46, 49, 52, 54, 55, 56, 57.
claim: 7, 19, 21, 23, 25, 28, 30, 33, 36, 41, 46, 49, 53, 55.
climbs: 46, 48, 50.
closedUnderMeet: 46, 48.
code: 16, 19, 21, 25, 28, 36.
col: 21.
cols: 20, 21.
cons: 11, 14, 15, 18, 24, 25.
consistent: 22, 24, 25.
context: 55.
count: 57.
covers: 39.
ctx: 11, 12, 16, 18, 22, 23, 26, 27, 31, 32, 33, 41, 57.
d: 26, 27, 28, 32, 56.
decisions: 53, 55.
delete: 51.
diff: 30, 42.
digit: 12, 14, 16, 18, 23, 26, 27, 32, 34, 42, 47.
do: 57.
dv: 34.
dw: 34.
e: 21, 26.
elems: 36, 37, 38, 39.
ell: 43, 46, 47, 48, 49, 50, 51.
err: 55.
Exit: 8.
f: 10, 11, 12, 15, 22, 30, 43, 44, 52, 53, 54, 55, 56, 57.
failures: 6, 7, 8.
fam: 9, 10, 11, 12, 43, 52, 54, 56, 57.
Fields: 10.
flag: 5.
Float64: 41.
fm: 52.
fmask: 11, 14, 15, 22.
fmt: 7, 8, 19, 21, 23, 28, 30, 33, 36, 41, 45, 53.
format: 7.
free: 11, 14, 24.
got: 21, 28.
horn: 43, 46, 47, 49.
i: 10, 13, 16, 21, 31, 46, 49, 53, 56, 57.
in: 36, 42.
IntN: 49, 50, 54, 57.
IntVar: 5.
isLattice: 36, 37, 38.
isStable: 11, 14, 15, 18, 23, 24, 25, 26, 28, 30, 33, 34, 35, 41.
Itoa: 18, 27, 55.
j: 21, 39, 50, 54, 56, 57.
join: 37, 38, 39.
k: 44, 50.
l: 16, 52, 55.
Len: 18, 27.
len: 10, 16, 26, 31, 36, 40, 46, 49, 57.
level: 55.
lhs: 39.
line: 21.
lit: 10, 14, 31, 44, 47, 54, 56, 57.
Lit: 52, 55.
Lookup: 55.
lowerSemi: 36, 37, 39.
lp: 34, 43.

lpp: 34.
ls: 55.
l7: 36, 38, 39.
m: 7, 32, 39, 46, 47, 50, 57.
main: 4.
make: 13, 14, 26, 30, 41, 57.
mark: 7.
maxN: 57.
meet: 32, 34, 35, 37, 38, 39.
mode: 5, 6, 7.
modular: 36, 37, 39.
more: 51.
n: 9, 10, 11, 12, 13, 14, 16, 18, 23, 26, 27, 32, 34, 41, 42, 43, 44, 47, 48, 49, 50, 51, 53, 54, 55, 56, 57.
nA: 11, 13, 14, 15, 19, 23, 26, 28, 30, 33, 35, 41, 47.
name: 27.
neg: 56.
New: 41, 49, 53, 55, 57.
newCtx: 12, 19, 21, 25, 28, 30, 36, 47, 57.
NewPCG: 41, 49, 53, 57.
next: 57.
nfam: 49.
nh: 33, 34.
nj: 41.
n3: 23.
ok: 7, 19.
on: 4, 7.
one: 25.
OnesCount32: 15, 52, 54.
os: 8.
out: 57.
p: 17, 18, 40, 41, 42.
Parse: 5.
parseFam: 9, 10, 19, 21, 25, 28, 30, 36.
pos: 56.
positive: 10, 47.
pow3: 11, 12, 13, 16, 24, 26, 32, 34.
printed: 43, 47.
Printf: 7, 8.
Println: 8, 19, 21, 23, 28, 30, 33, 36, 41, 45, 53.
q: 17, 18, 40.
queue: 26.
r: 21.
rand: 41, 49, 53, 57.
reason: 3.
rebuilds: 46, 47, 49.
reps: 5, 6, 23, 33, 41, 49, 53.
rg: 49, 50.
rng: 41, 57.
rows: 20, 21.
rt: 53, 54.
runs: 53, 55.
s: 10, 16, 30, 31, 46, 55.
sanity: 3.
sat: 52, 55.
seed: 5, 6, 41, 49, 53, 57.
seen: 26, 28, 35, 57.
setCode: 30, 31.
SetProof: 3.
SetTrace: 3, 52, 55.
Solve: 55.
Sprintf: 7, 21.
src: 43, 44.
strconv: 18, 27, 55.
String: 18, 19, 21, 27.
strings: 10, 18, 27.
StringVar: 5.
sum: 42.
sweep: 23, 33, 41, 57.
t: 15, 18, 40.
target: 28.
term: 17, 18, 40.
tloc: 3.
tm: 52.
tmask: 11, 14, 15, 22.
trail: 52, 55.
TrailingZeros32: 15.
trailStable: 52, 55.
twelve: 29, 30, 46.
u: 38, 39.
Uint64Var: 5.
unstable: 53, 55.
up: 48, 51.
v: 10, 12, 14, 16, 18, 23, 24, 26, 27, 32, 34, 39, 40, 41, 42, 43, 44, 47, 48, 50, 51, 54, 55, 57.
value: 3, 40, 42.
w: 10, 34, 39.
waerden: 53, 56.
want: 30, 42.
weight: 18, 19, 21, 42.
WriteByte: 27.
WriteString: 18, 27.
x: 25, 35, 40, 42.
xp: 24, 25.
xpp: 24.

y: 35.
zero: 17, 18, 19, 40.

⟨ Add the clause that forces v 44 ⟩ Used in section 43
⟨ Build a random family that satisfies the hypotheses 50 ⟩ Used in section 49
⟨ Check part (a) against Algorithm C 53 ⟩ Used in section 4
⟨ Check part (c) 23 ⟩ Used in section 4
⟨ Check part (h) at a 34 ⟩ Used in section 33
⟨ Check part (i) at every target 35 ⟩ Used in section 33
⟨ Check parts (b) and (e) 19 ⟩ Used in section 4
⟨ Check parts (d) and (f) 28 ⟩ Used in section 4
⟨ Check parts (h) and (i), and the lattice of part (d) 33 ⟩ Used in section 4
⟨ Check the construction of part (g) 45 ⟩ Used in section 4
⟨ Check the counterexample the answer gives 25 ⟩ Used in section 23
⟨ Check the identity of part (j) 41 ⟩ Used in section 4
⟨ Check the table of part (e) 21 ⟩ Used in section 19
⟨ Check the twelve sets of part (f) 30 ⟩ Used in section 28
⟨ Compare the sum below a with the product of its p's 42 ⟩ Used in section 41
⟨ Compare the three statements with unconstrainedness 24 ⟩ Used in section 23
⟨ Decide stability and constrainedness 15 ⟩ Used in section 12
⟨ Decide whether the elements form a lattice 37 ⟩ Used in section 36
⟨ Drop the members that cannot climb 51 ⟩ Used in section 50
⟨ Find the join of a and b 38 ⟩ Used in section 37
⟨ Functions 7 ⟩ Used in section 4
⟨ Global variables 6 ⟩ Used in section 4
⟨ Look at the lattice that part (d) mentions 36 ⟩ Used in section 33
⟨ Make a random 3SAT problem 54 ⟩ Used in section 53
⟨ Read the command line 5 ⟩ Used in section 4
⟨ Say how it went 8 ⟩ Used in section 4
⟨ Set up the powers of three 13 ⟩ Used in section 12
⟨ Solve it, watching every decision 55 ⟩ Used in section 53
⟨ Tabulate the masks 14 ⟩ Used in section 12
⟨ Test lower semimodularity and modularity 39 ⟩ Used in section 37
⟨ Try both readings on random families 49 ⟩ Used in section 45
⟨ Try both readings on the family of part (f) 46 ⟩ Used in section 45
⟨ Types 9 ⟩ Used in section 4

Stable partial assignments

	Section	Page
Introduction	1	1
Clauses and partial assignments	9	4
Weights	17	7
Stability, one variable at a time	22	9
Climbing	26	11
The lattice	32	13
The identity of part (j)	40	18
Definite Horn clauses for a given family	43	20
Algorithm C's trail	52	25
Random problems	57	28
Index	58	29